When nobody reads the code, the review is a score against the real thing. A check you have never seen fail is not evidence.

Friday, November 13, 2026

AWS shipped a local copy of its cloud that agents wrote and merged with no engineer reading the code, and published how often it agrees with real AWS. In the same week, DeepMind proved its proof checker could catch planted bugs, a new benchmark tested its graders against lazy and sabotaged patches before scoring anyone, and a scan of public repositories found the files coding agents write to remember approvals leaking passwords. This week's daily editions tracked the other half of that story: one fail-open path after another closed in agent sandboxes.

A note on sourcing: arXiv, AWS, DeepMind, GeekWire, and several blogs could not be opened from this week's research environment, so the summaries below rest on abstracts and search extracts of the primary pages, not full texts. Where coverage disagreed, I say so.

THE STORY OF THE WEEK

AWS ships an emulator its agents wrote, and the acceptance test is real AWS

On Nov 11 AWS released AWS Emulator, a free local stand-in for more than 200 AWS services, built from more than 400 public API models, that runs in CI without an AWS account. AWS says every line was written by autonomous agents running on Bedrock AgentCore, and GeekWire reports that its engineers neither wrote nor reviewed the code. Matt Goldberg, the senior principal engineer behind it, started with one prompt that produced a first SQS emulator in about 15 minutes. Making it trustworthy took the rest.

The interesting part is the process, not the scale. Hundreds of agents work in three teams. One team writes fidelity tests from the public API models and corrects each test until it passes against real AWS, so every test records how AWS actually behaves. A second team replays those tests against the emulator and files an issue for each gap. A third team fixes one issue per agent. Every change then passes a merge gate: compilation, coding standards, several adversarial security-review agents plus a general reviewer, and a fidelity regression check that blocks any change that makes existing fidelity worse. A test passes only if status code, response body, exact error code and message, and the resulting resources all match. Operations that are not built return HTTP 501 so tests fail visibly instead of passing on a stub.

And AWS published the score. Release v1.0.322 matches real AWS on roughly 24,243 of about 42,900 fidelity tests (coverage cites both 56.5% and 56.6%, from slightly different test totals), from above 90% for KMS to under 20% for CloudFormation. The limits are stated too: the emulator does not evaluate IAM or resource policies the way AWS does, does not simulate throttling or outages, and AWS says not to use it for security testing. An independent test found it refused more unauthorized calls than its docs claim, while other testers found requests real AWS would deny that succeed locally.

What we can learn: Swami Sivasubramanian's guest post on All Things Distributed puts the engineer's job as deciding what to build and writing the specifications, tests, and merge gates that define correct. That is a QA job description. Two details make it credible rather than hopeful: every test was proven against the real system before it was allowed to judge anything, and the result ships with a per-service score instead of a claim that it works. It is vendor-reported and one release old.

Takeaway: For any agent-built component a client wants to accept without line-by-line review, ask for three things: tests proven against a real reference, a gate that refuses regressions, and a published score per area. Missing any one, the review has to stay.

THREE READS WORTH YOUR WEEKEND

Introducing AlphaProof Code

Google DeepMind, Nov 10, with a technical page and open-source Lean development

A multi-agent system proved forward secrecy, post-compromise security, and authentication for libsignal's PQXDH handshake and Triple Ratchet,

against the compiled AArch64 binary of libsignal v0.105.4 rather than the Rust source. People and models drafted the specifications and experts reviewed each one, Gemini-powered agents wrote the Lean proofs, and Lean's kernel checked every proof, so nobody has to trust the agent's reasoning. The work reportedly took about four weeks, with roughly 1% routed to a person.

Most interesting: the team reinserted two previously fixed libsignal bugs, and the system proved those builds violate the properties. That is a test of the checker, not of the code. Coverage notes the proofs still rest on about 26,000 lines of trusted specifications and machine semantics, and the line counts quoted for the Lean development differ between sources.

Why it matters: The pattern transfers without formal methods: a checker the agent cannot argue with, human attention spent on the specification, and a planted bug that proves the checker can fail.

AWS Emulator and the engineer's job when agents write the code

Swami Sivasubramanian, guest post on All Things Distributed, Nov 11

The companion essay to the story above. Sivasubramanian argues that agents now write code faster than any team can check it in production, so the engineer's work moves to defining acceptable behavior and automating its enforcement: specifications, fidelity tests, and the merge gate, including deterministic checks that keep agents out of code they should not touch.

Most interesting: the essay treats the merge gate as the product the engineers own. A useful outside view is Qodo's response, which says engineers still decide which findings need a person; Qodo sells review tooling, so read it as a vendor's framing.

Why it matters: It is a senior executive saying out loud that the gate, not the reviewer, is where correctness lives. That raises the bar for how gates are tested.

Hide your slop PR descriptions behind <details><summary>

Simon Willison, Nov 9

Willison notes that most changes to his own projects now arrive as pull requests opened for him by Claude Code or Codex. His small, practical suggestion: keep the long agent-written description, but collapse it inside a details block labeled with the model that wrote it, and put a short human summary on top.

Most interesting: it is a disclosure convention a team can adopt on Monday. The reviewer sees the human intent first, and the record of which model wrote what stays in the pull request.

Why it matters: Read it next to the AI slop study below, where only 14% of flagged pull requests disclosed AI use. Disclosure that costs nothing is the cheapest trust signal a reviewer can get.

FROM THE LAB

ExpertCrowd-SWE: Measuring Frontier Coding Agents on Expert-Authored Engineering Tasks in Unfamiliar Codebases

Vrdoljak et al. (Toptal ExpertCrowd) | preprint, Nov 9

100 multi-file change requests written by practicing engineers across 87 open-source projects and 11 languages, each pinned to a commit with a sealed reference fix and a partial-credit grader. The grader is calibrated before any agent is scored: a task only ships if the reference fix scores exactly 1.0 and both the untouched repository and an inert change score 0. Nearly half the tasks also carry control patches: alternative valid fixes that must pass, and lazy or adversarial patches that must fail or score low. Must-pass blockers cap the score for a change a maintainer would refuse. Across seven models no task is solved by all of them, and scores span 35 points. 76 tasks are held out against contamination, and the judge is a single model for every task.

For practice: Before an automated check is allowed to accept agent work, feed it a no-op, a lazy fix, and a sabotaged fix, and confirm it rejects all three.

Replay Fidelity in LLM Agent Evaluation: Randomness, Hidden State, and Test Oracles in AppWorld

Liu, Lu | preprint, Nov 8

The authors replayed all 71 failed runs in a historical AppWorld corpus, 284 isolated replays in total, to see whether a failure can be reproduced faithfully. Rebuilding the declared settings reproduced 42 cases; also resetting the random-number generator before each interaction reproduced 68. Checking database changes and the native test oracle narrowed the disputed cases from

26 to four. In four cases outputs and verdicts matched but the stored state did not, and in one, identical API calls returned matching responses with different row effects.

For practice: A replayed agent failure is only evidence if randomness and state are pinned, and a matching output is not a matching outcome. Compare resulting state, not just printed results, when you debug or re-grade agent runs.

An Empirical Study of Maintainer-Identified AI Slop Pull Requests in Free and Open Source Software Projects

Raskin, Devroey, Vanderose | preprint, Nov 11

A study of pull requests that maintainers themselves flagged as AI slop. Flagged pull requests rose from mid-2025 and peaked in May 2026; they are mostly additive, rarely merged, and usually closed within three days. Authors disclosed AI use in 14% of cases, 1% only after a maintainer asked. Most were closed with no stated reason, and the reasons given ranged from concrete build failures to "looks AI to me". The authors read this as maintainers' willingness to engage eroding into fast rejection and automated filtering. The full sample size was not in the extracts I could read.

For practice: "Looks AI to me" is not a gate. Teams receiving agent pull requests need stated, checkable acceptance criteria, or they will reject good changes and accept bad ones by mood.

FROM THE FIELD

104,185 public settings files show what agents leave behind

Security firm Apiiro scanned every public copy of Claude Code's `.claude/settings.local.json` on GitHub between Nov 1 and 3. When a developer answers "Yes, and don't ask again", Claude Code saves the approved command as an allow rule, word for word, so a password typed into that command is saved too. Of 104,185 copies in 103,617 repositories, 1,122 held at least one real secret, 1,379 distinct credentials in all, including SSH logins, GitHub and npm tokens, and AWS keys (The Hacker News, Cybernews). Counting git history, shared settings, and chat logs, the total reaches 2,539 secrets across 1,994 repositories.

The QA-relevant number: TruffleHog flagged 57% of the affected files and gitleaks 50%, and neither flagged 215, because inline passwords such as `PGPASSWORD=` have no known key format. Apiiro says Claude Code was the only one of 16 agents it reviewed that writes approved commands into project files by default. The Cloud Security Alliance research note (Nov 8) traces the behavior to a GitHub issue filed in August 2025. Anthropic said it is looking at secret scrubbing; no fix appears in the release notes through 2.1.322. Apiiro did not test whether the keys still work.

The lesson is broader than one tool: agents write working files (approvals, memory, transcripts) into the repository, and a green secret scan that was never pointed at them proves nothing.

THE QA LENS

Test the check before you trust the pass

Every strong result this week tested its checker before trusting it. AWS corrected each fidelity test against real AWS before it judged the emulator. ExpertCrowd-SWE made its graders reject an inert change and score lazy and adversarial patches correctly before scoring any model. DeepMind planted two known bugs to prove its proofs could fail. OpenAI's new Daybreak Red Team reruns a working exploit after a fix to confirm the fix holds, which turns the exploit into a regression test (vendor-described, no benchmarks published).

The failures went the other way: secret scanners that passed while missing a fifth of the leaked files, and, in the daily editions, sandboxes that ran unsandboxed when their settings were invalid. A check nobody has seen fail is an assumption.

QA's opportunity is to own the known-bad inputs: a planted bug for each critical property, a no-op and a sabotaged patch for each acceptance gate, a corrupt config for each sandbox, and a fake inline password for each secret scan, run on a schedule, with the results reported next to the pass rate.

ONE BIG IDEA

> ***When nobody reads the code, the review is a score against the real thing. A check you have never seen fail is not evidence.***

WEEKEND RABBIT HOLE

Point your own acceptance gate at ExpertCrowd-SWE's control patches

What it is: the public ExpertCrowd-SWE subset on Hugging Face releases 24 tasks with their environments, graders, and results, including the reference fixes and, for many tasks, lazy and adversarial patches the grader must reject.

Why it was selected: most teams have never measured whether their own agent acceptance gate (CI tests, an AI reviewer, an LLM judge) can tell a real fix from a plausible fake. These tasks come with known answers, so you can run your gate on them and count how many bad patches it lets through.

Pay attention to: the benchmark uses one model as judge for every task, the leaderboard is hosted by the company that built it, and 24 public tasks is a small sample. Use it to test your gate, not to rank models.

FUTURE BET

BET: By the end of 2027, at least two major software vendors will ship agent-built components with a published pass rate against a reference system in place of line-by-line human code review.

WHY: AWS has now shipped a 200-service product accepted on fidelity tests against real AWS, with per-service scores published and no human code review. DeepMind's AlphaProof Code shows the same shift for security properties: experts review the specification, and a checker accepts the proof.

SOURCE: AWS Emulator launch (Nov 11), Sivasubramanian on All Things Distributed (Nov 11), Google DeepMind AlphaProof Code (Nov 10)

Bet check. *By the end of 2027, at least one major AI code review product will publish its defect-detection rate on a public, labeled benchmark next to its comment-usefulness numbers:* **Holding.** No new publication this week. Several review vendors already report detection rates, but on benchmarks they built and graded themselves, which does not meet the bet's bar. ExpertCrowd-SWE's control patches show what a credible labeled set looks like.

RADAR CHECKPOINT

Decision: promote **Sandbox negative testing** to Production now, and add **Agent config secret scanning** (Governance, Production now). Sandbox negative testing was touched on Tuesday, Wednesday, and Friday: Claude Code 2.1.321 and 2.1.323 fixed a read-only command that ran outside the sandbox, deny rules that missed cp, mv, sudo, and env, and headless runs that went unsandboxed when sandbox settings failed validation, and Copilot CLI 1.0.110 stopped an invalid settings file from resetting the sandbox to defaults. Each was a fail-open path that only an escape test would have found, and the tests are cheap, so they belong in every agent pipeline now. Agent config secret scanning enters at Production now because the fix uses scanners teams already run, pointed at agent folders, with rules for inline passwords and a sweep of git history. Evidence refreshes without moves: Claude Code's deny-instead-of-prompt setting and Copilot CLI's warning when policy overrides `--yolo` join Fail-closed agent approvals, ExpertCrowd-SWE joins Independent agent quality gates, AlphaProof Code joins Adversarial test validation, and AWS Emulator joins Self-testing agent loops. Fidelity-scored service emulators stay a candidate (one launch, about 57% overall fidelity, no IAM evaluation), and so does an external agent stop control (OpenAI's Control Room covers only its internal research workloads).

View the current SDLC Radar